Marketplace to Customer School: End-to-End Operations Guide

Status: operational guide
Last verified: 2026-09-05
Scope: Wattanam Marketplace, Platform Admin, and one isolated Railway project per customer school

This guide documents two supported workflows:

1. **Manual:** use Railway Dashboard and Platform Admin.
2. **CLI:** use Railway CLI plus the public Platform and school APIs.

Both workflows produce the same architecture:

```
Wattanam Marketplace Production
├── Postgres
├── marketplace-api        public, port 3010
└── platform-admin-web     public, port 8080

Wattanam - <School> - Production
├── Postgres               private
├── api                    private, port 3001, /data volume
├── web                    private, port 3000
└── gateway                public, port 8080
```

Never place two schools in one Railway project or database. Only the school gateway is public; the school's API, web service, and database remain private.

---

1. Information to prepare

Prepare these values before starting:

Value	Example
Marketplace project	`Wattanam Marketplace Production`
Marketplace origin	`https://marketplace-api-production-xxxx.up.railway.app`
Platform Admin origin	`https://platform-admin-web-production-xxxx.up.railway.app`
School legal name	`Bora School`
Immutable school slug	`bora-school`
Owner name/email	`Bora` , `owner@example.com`
Locale/timezone/currency	`km-KH` , `Asia/Phnom_Penh` , `USD`
Railway workspace	workspace name or ID
Internal reason	contract, ticket, or approved onboarding reason

Use a password manager for passwords, signing keys, recovery codes, and Railway tokens. Do not put them in Git, screenshots, tickets, chat, or this document.

---

2. Marketplace and Platform Admin prerequisites

Complete this section once. Every customer school connects to this central control plane.

2.1 Manual marketplace deployment

1. Railway Dashboard -> **New Project** -> **Empty Project**.
2. Rename it **Wattanam Marketplace Production**.
3. Add PostgreSQL.
4. Add the GitHub repository as service `marketplace-api` .
5. Keep repository root blank and set Dockerfile path to `apps/marketplace-api/Dockerfile` .
6. Add a volume mounted at `/data/artifacts` .
7. Generate a public domain on port `3010` .
8. Configure at least:

```
NODE_ENV=production
DATABASE_URL=${{Postgres.DATABASE_URL}}
PORT=3010
PUBLIC_BASE_URL=https://<marketplace-domain>
ARTIFACT_STORAGE_PATH=/data/artifacts
METADATA_SIGNING_KEY_ID=<key-id>
METADATA_SIGNING_PRIVATE_KEY=<sealed Ed25519 private key>
ENTITLEMENT_SIGNING_KEY_ID=<different-key-id>
ENTITLEMENT_SIGNING_PRIVATE_KEY=<different sealed Ed25519 private key>
IDENTITY_ENCRYPTION_ACTIVE_KEY_ID=<identity-key-id>
IDENTITY_ENCRYPTION_KEYS_JSON=<sealed JSON keyring>
MARKETPLACE_ADMIN_TOKEN=<sealed random secret>
PAYMENT_WEBHOOK_SECRET=<sealed random secret>
```

Production normally requires transactional email. If email is deliberately unavailable during a pilot, use the repository's explicitly guarded MFA-only configuration and record that email verification and password recovery are unavailable. Do not use fake provider credentials.

9. Add the repository again as `platform-admin-web`.
10. Keep repository root blank and set Dockerfile path to `infrastructure/railway/platform-admin.Dockerfile`.
11. Set `MARKETPLACE_API_URL` to the private marketplace address, for example `http://marketplace-api.railway.internal:3010`.
12. Generate its public domain on port `8080`.
13. Wait for both deployments to reach **Success**.

Create the first Platform Owner only through the audited bootstrap script. The production image does not expose public operator signup. On first login, enroll MFA and save the recovery codes.

Full marketplace secret-generation and bootstrap details are in railway-manual-setup-guide.md.

2.2 CLI marketplace deployment

Install and authenticate Railway CLI:

```
npm install --global @railway/cli
railway login
railway whoami
```

From the repository root:

```
$workspace = '<RAILWAY_WORKSPACE_ID>'
$marketplaceProject = railway init `
  --name 'Wattanam Marketplace Production' `
  --workspace $workspace `
  --json | ConvertFrom-Json

railway add --database postgres --json
railway add --service marketplace-api --json
railway add --service platform-admin-web --json

railway service link marketplace-api
railway volume add --mount-path /data/artifacts --json
railway domain --service marketplace-api --port 3010 --json
railway domain --service platform-admin-web --port 8080 --json
```

Set `RAILWAY_DOCKERFILE_PATH=apps/marketplace-api/Dockerfile` on `marketplace-api` and `RAILWAY_DOCKERFILE_PATH=infrastructure/railway/platform-admin.Dockerfile` on `platform-admin-web`. Set secrets with stdin so they do not appear in shell history:

```
Get-Content -Raw '<secret-file>' |
  railway variable set METADATA_SIGNING_PRIVATE_KEY --stdin `
    --service marketplace-api --skip-deploys --json
```

Set the non-secret variables listed in section 2.1, then deploy both services:

```
railway up . --path-as-root --service marketplace-api --environment production --detach
railway up . --path-as-root --service platform-admin-web --environment production --detach
railway service list --json
```

Do not proceed until both health checks pass:

```
curl.exe --fail-with-body https://<marketplace-domain>/health/live
curl.exe --fail-with-body https://<marketplace-domain>/health/ready
```

---

3. Workflow A: manual customer-school onboarding

This is the recommended process for routine operator onboarding.

3.1 Create the directory record first

1. Sign in to Platform Admin using the operator email, password, and current MFA code.
2. Open **Customer schools**.
3. Select **Create customer school**.

4. Enter the legal name, immutable slug, contact, region, plan, timezone, locale, currency, and an internal reason.
5. Leave Railway project ID and gateway origin empty if infrastructure does not exist yet.
6. Submit. The new record starts as `PROSPECT`.
7. Open the school detail and choose **Create enrollment token**.
8. Copy the plaintext token immediately into a password manager. It is displayed once, stored only as a hash, expires, and can be redeemed once.

3.2 Create the isolated Railway project

1. Railway Dashboard -> **New Project** -> **Empty Project**.
2. Rename it `Wattanam - <School Name> - Production`.
3. Add PostgreSQL.
4. Add the same GitHub repository three times, naming the services `api`, `web`, and `gateway`.
5. Configure roots:

Service	Root directory	Dockerfile
`api`	`backend`	`Dockerfile`
`web`	`frontend`	`Dockerfile`
`gateway`	`infrastructure/gateway`	`Dockerfile`

6. Add an API volume mounted at `/data`.
7. Generate a domain only for `gateway`, targeting port `8080`.
8. Do not generate public domains for `api`, `web`, or PostgreSQL.

3.3 Configure school services

Configure `api`:

```
NODE_ENV=production
PORT=3001
DATABASE_URL=${{Postgres.DATABASE_URL}}
JWT_SECRET=<unique random value of at least 32 characters>
CORS_ORIGINS=https://<school-gateway-domain>
PUBLIC_APP_URL=https://<school-gateway-domain>
COOKIE_SECURE=true
PROCESS_ROLE=combined
BACKUP_DIR=/data/backups
PLUGIN_DIR=/data/plugins
PLUGIN_DATA_DIR=/data/plugin-data
INSTALLATION_KEY_DIR=/data/marketplace-identity
MARKETPLACE_URL=https://<marketplace-domain>
UPDATE_REPOSITORY_KEY_ID=<active marketplace repository key ID>
UPDATE_REPOSITORY_PUBLIC_KEY=<pinned marketplace public key>
```

Retrieve the public repository key—never the private key—from:

```
curl.exe --fail-with-body https://<marketplace-domain>/v1/repository-key
```

Configure `web`:

```
NODE_ENV=production
PORT=3000
API_URL=http://${{api.RAILWAY_PRIVATE_DOMAIN}}:3001
MARKETPLACE_URL=https://<marketplace-domain>
NEXT_PUBLIC_API_URL=
```

Configure `gateway`:

```
PORT=8080
API_UPSTREAM=${{api.RAILWAY_PRIVATE_DOMAIN}}:3001
WEB_UPSTREAM=${{web.RAILWAY_PRIVATE_DOMAIN}}:3000
```

Redeploy the gateway after Railway has resolved both referenced private domains. A gateway log showing `dial tcp :3000` means it captured the reference too early; redeploying refreshes it.

3.4 Install and enroll the school

1. Open `https://<school-gateway-domain>/setup`.
2. Enter the exact directory name and slug, plus locale, timezone, currency, owner name, owner email, and a strong unique password.
3. Complete setup once. Setup locks after installation.
4. Sign in as the school `SUPER_ADMIN`.

5. Open **Admin -> Plugins** or the marketplace installation section.
6. Register the installation using the one-time enrollment token from Platform Admin.
7. Return to Platform Admin and refresh the school detail.

Expected result:

- lifecycle is `ACTIVE`;
- Railway project ID and gateway origin are recorded;
- installation ID is claimed;
- exactly one active installation public key is displayed;
- the plaintext enrollment token is no longer available.

---

4. Workflow B: CLI customer-school onboarding

The following commands use PowerShell and intentionally prompt for credentials. Do not replace the prompts with plaintext secrets in committed scripts.

4.1 Authenticate to the Platform API

```
$marketplace = 'https://<marketplace-domain>'
$operatorEmail = Read-Host 'Platform operator email'
$operatorPassword = Read-Host 'Platform operator password'
$mfa = Read-Host 'Current MFA or recovery code'

$login = Invoke-RestMethod -Method Post `
  -Uri "$marketplace/platform/v1/auth/sessions" `
  -ContentType 'application/json' `
  -Body (@{
    email = $operatorEmail
    password = $operatorPassword
    code = $mfa
  } | ConvertTo-Json)

if ($login.status -ne 'authenticated') {
  throw "Platform login did not authenticate: $($login.status)"
}

$platformHeaders = @{
  Authorization = "Bearer $($login.sessionToken)"
  'x-correlation-id' = "school-cli-$([guid]::NewGuid())"
}
```

4.2 Create the directory record and enrollment token

```
$schoolName = 'Bora School'
$schoolSlug = 'bora-school'

$school = Invoke-RestMethod -Method Post `
  -Uri "$marketplace/platform/v1/schools" `
  -Headers $platformHeaders `
  -ContentType 'application/json' `
  -Body (@{
    name = $schoolName
    slug = $schoolSlug
    region = 'asia-southeast1'
    timezone = 'Asia/Phnom_Penh'
    locale = 'km-KH'
    currency = 'USD'
    contactName = 'Bora'
    contactEmail = 'owner@example.com'
    internalReason = 'Approved customer onboarding ticket'
  } | ConvertTo-Json)

$claim = Invoke-RestMethod -Method Post `
  -Uri "$marketplace/platform/v1/schools/$($school.id)/enrollment-tokens" `
  -Headers $platformHeaders `
  -ContentType 'application/json' `
  -Body (@{
    lifetimeDays = 1
    internalReason = 'Claim freshly provisioned school installation'
  } | ConvertTo-Json)

$enrollmentToken = $claim.token
```

Keep `$enrollmentToken` only for this terminal session. Do not print or save it.

4.3 Create Railway services

From the repository root:

```
$workspace = '<RAILWAY_WORKSPACE_ID>'
$project = railway init `
  --name "Wattanam - $schoolName - Production" `
  --workspace $workspace `
  --json | ConvertFrom-Json

$schoolProjectId = $project.id
railway add --database postgres --json
railway add --service api --json
railway add --service web --json
railway add --service gateway --json

railway service link api
railway volume add --mount-path /data --json
$domainResult = railway domain --service gateway --port 8080 --json | ConvertFrom-Json
$gatewayOrigin = $domainResult.domain
```

Generate and set the school JWT without printing it:

```
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$bytes = New-Object byte[] 48
$rng.GetBytes($bytes)
$rng.Dispose()
$jwt = [Convert]::ToBase64String($bytes).TrimEnd('=').Replace('+','-').Replace('/','_')
$jwt | railway variable set JWT_SECRET --stdin --service api --skip-deploys --json | Out-Null
```

Set the non-secret API, web, and gateway variables from section 3.3. Record the Railway project and gateway in Platform Admin:

```
Invoke-RestMethod -Method Post `
  -Uri "$marketplace/platform/v1/schools/$($school.id)/infrastructure" `
  -Headers $platformHeaders `
  -ContentType 'application/json' `
  -Body (@{
    railwayProjectId = $schoolProjectId
    gatewayOrigin = $gatewayOrigin
    internalReason = 'Record newly provisioned Railway infrastructure'
  } | ConvertTo-Json)
```

Deploy in dependency order:

```
railway up .\backend --path-as-root --service api --environment production --detach
railway up .\frontend --path-as-root --service web --environment production --detach
railway up .\infrastructure\gateway --path-as-root --service gateway --environment production --
detach
railway service list --json
```

Wait until Postgres, `api`, `web`, and `gateway` all show `SUCCESS`.

4.4 Install through the API

```
$ownerPassword = Read-Host 'Temporary school-owner password'

$installation = Invoke-RestMethod -Method Post `
  -Uri "$gatewayOrigin/api/installation/install" `
  -ContentType 'application/json' `
  -Body (@{
    schoolName = $schoolName
    schoolSlug = $schoolSlug
    locale = 'km-KH'
    timezone = 'Asia/Phnom_Penh'
    currency = 'USD'
    ownerName = 'Bora'
    ownerEmail = 'owner@example.com'
    ownerPassword = $ownerPassword
  } | ConvertTo-Json)

if ($installation.state -ne 'installed') { throw 'School installation did not complete' }
```

The install endpoint is available only before installation and is transactionally locked. A repeat install must be rejected.

4.5 Redeem the token through the signed installation flow

```
$schoolLogin = Invoke-RestMethod -Method Post `
  -Uri "$gatewayOrigin/api/auth/login" `
  -ContentType 'application/json' `
  -Body (@{
    email = 'owner@example.com'
    password = $ownerPassword
  } | ConvertTo-Json)

$schoolHeaders = @{ Authorization = "Bearer $($schoolLogin.access_token)" }

$registration = Invoke-RestMethod -Method Post `
  -Uri "$gatewayOrigin/api/admin/marketplace/installation/register" `
  -Headers $schoolHeaders `
  -ContentType 'application/json' `
  -Body (@{
    label = "$schoolName Production"
    enrollmentToken = $enrollmentToken
  } | ConvertTo-Json)
```

The school creates its Ed25519 private key under `/data/marketplace-identity`. Only the public key and fingerprint reach the marketplace. Registration consumes the enrollment token and changes the directory lifecycle to `ACTIVE`.

Clear sensitive shell variables when finished:

```
$operatorPassword = $null
$mfa = $null
$ownerPassword = $null
$enrollmentToken = $null
$login = $null
$schoolLogin = $null
```

---

5. Production release versus source pilot

`tools/provision-railway-school.js` is the production provisioner. It refuses mutable image tags and requires a release blueprint containing digest-pinned `api`, `web`, and `gateway` images.

Use it after CI has published the images and produced the blueprint:

```
node tools/provision-railway-school.js `
  --blueprint .\release\school-blueprint.json `
  --school-slug bora-school `
  --school-name 'Bora School'

node tools/provision-railway-school.js `
  --blueprint .\release\school-blueprint.json `
  --school-slug bora-school `
  --school-name 'Bora School' `
  --workspace '<RAILWAY_WORKSPACE_ID>' `
  --execute `
  --confirm-slug bora-school
```

The first command is a dry run. Review it before using `--execute`. Direct `railway up` deployment from sections 2 and 4 is suitable for a controlled source pilot, but it is not an immutable, reproducible production release.

---

6. Final verification

Run these checks from a machine outside Railway:

```
curl.exe --fail-with-body "$gatewayOrigin/gateway-health"
curl.exe --fail-with-body "$gatewayOrigin/api/version"
curl.exe --fail-with-body "$gatewayOrigin/api/health/ready"
curl.exe --fail-with-body "$gatewayOrigin/api/installation/status"
curl.exe -sS -o NUL -w '%{http_code}' "$gatewayOrigin/login"
```

Acceptance requires:

- gateway health is `200`;
- API readiness reports database `up`;
- installation state is `installed` with the expected name and slug;
- login returns `200`;
- Platform Admin shows the school as `ACTIVE` and claimed;
- Railway project ID, gateway origin, and installation ID match;
- one active installation key exists;
- API redeploy preserves the installation identity;
- no internal service has a public domain;
- backup and restore have been tested before production handoff.

7. Troubleshooting

Symptom	Cause	Resolution
Platform login says `Invalid request`	Extra text/newline pasted as the password, or malformed request	Clear the field and paste only the password; direct API should return `mfa_required` before code entry
Marketplace API is unavailable	Wrong `MARKETPLACE_API_URL` or failed API service	Use the marketplace private domain from Platform Admin and verify `/health/ready`
Gateway returns `502` and logs `dial tcp :3000`	Gateway deployment captured an unresolved web reference	Confirm `WEB_UPSTREAM=web.railway.internal:3000`, then redeploy gateway
Registration fails with `/data/marketplace-identity` `EACCES`	API image did not initialize Railway volume ownership	Deploy the current backend Dockerfile, which creates/chowns `/data` before dropping to `node`
School remains `AWAITING_CLAIM`	Token was not redeemed or registration failed	Correct the school API, then retry the same unconsumed token before expiry
Token was lost	Plaintext is intentionally unrecoverable	Revoke the live token in Platform Admin and mint a replacement
Duplicate project/gateway error	Infrastructure already belongs to another directory record	Stop and reconcile; never bypass the uniqueness check

8. Operator handoff

Give the school owner only:

- school gateway URL;
- their owner email;
- a one-time temporary password through an approved secure channel;
- instructions to change the password immediately.

Do not give a school owner marketplace signing keys, Railway project tokens, enrollment tokens, database credentials, Platform Admin credentials, or another school's information.

Related documents:

- Complete Platform CLI Guide
- Manual Railway Setup Guide
- Customer School Provisioning Control Plan
- Per-school Railway Architecture